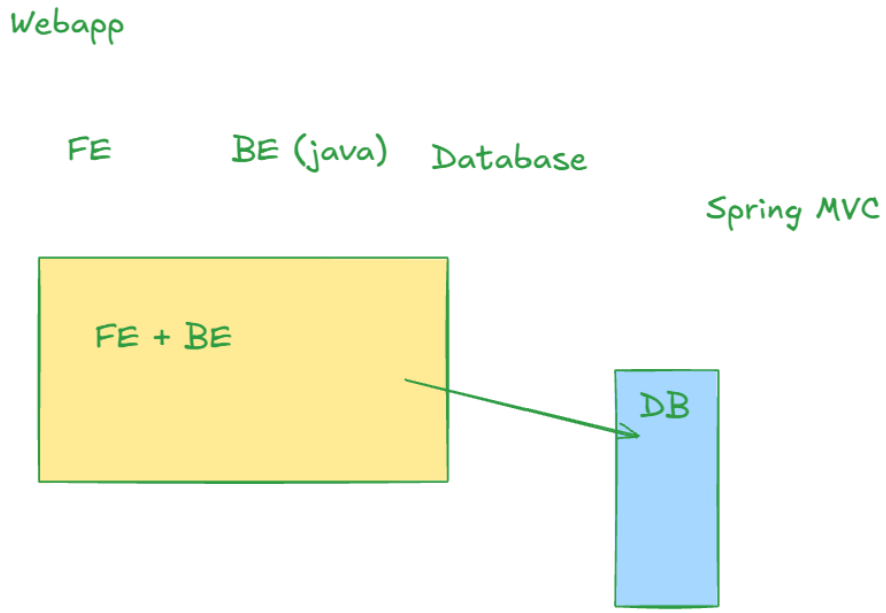
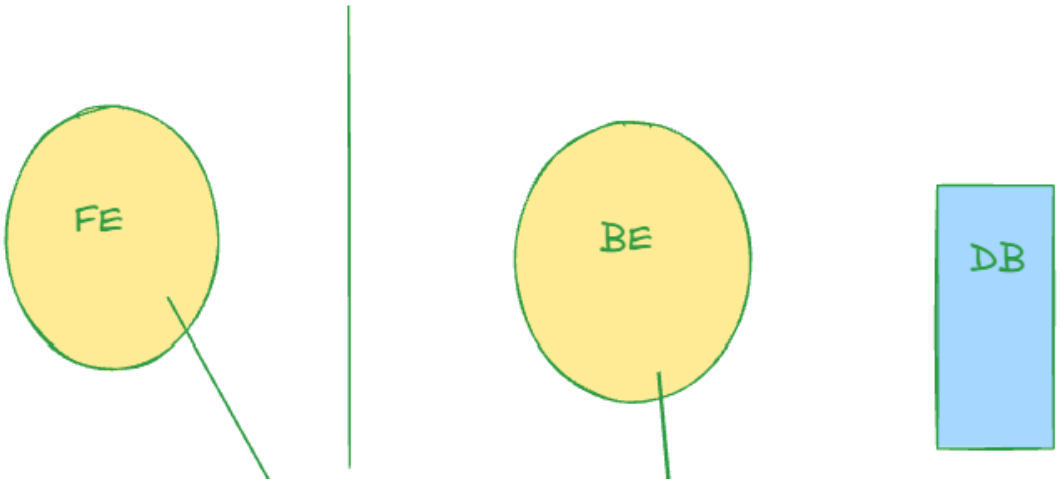
LIVE NOTES

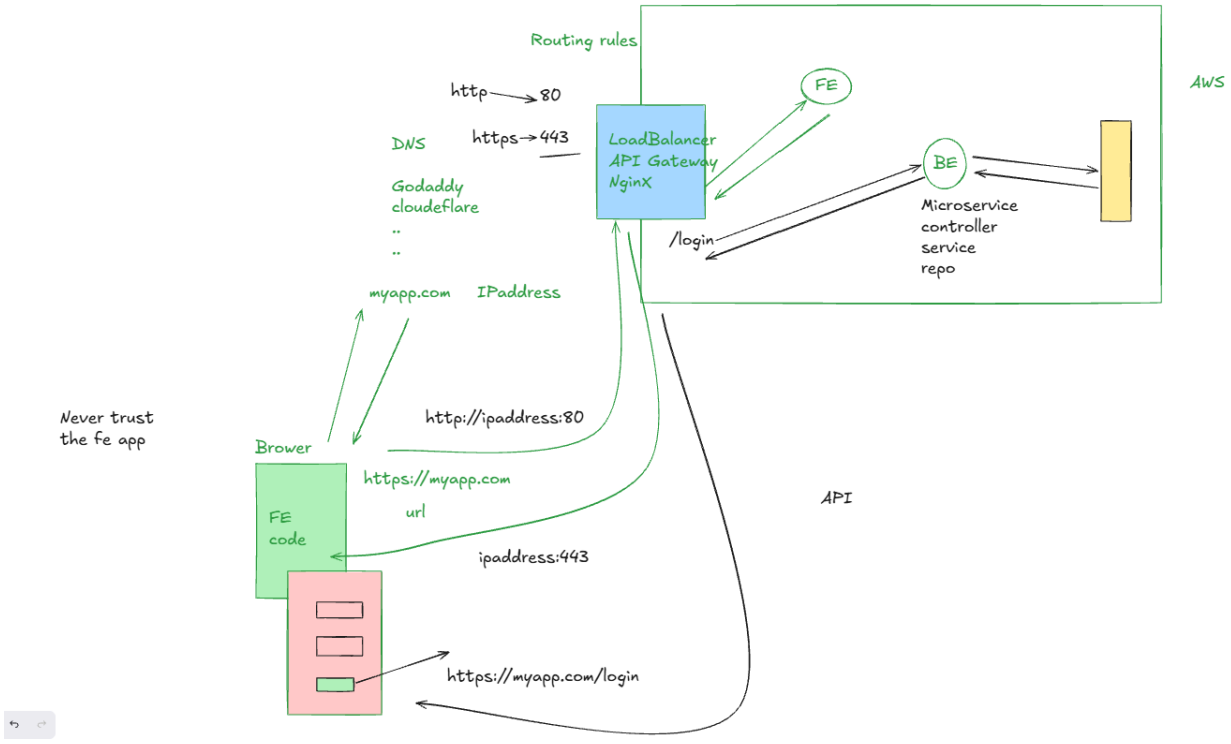
Earlier approach frontend-backend both together



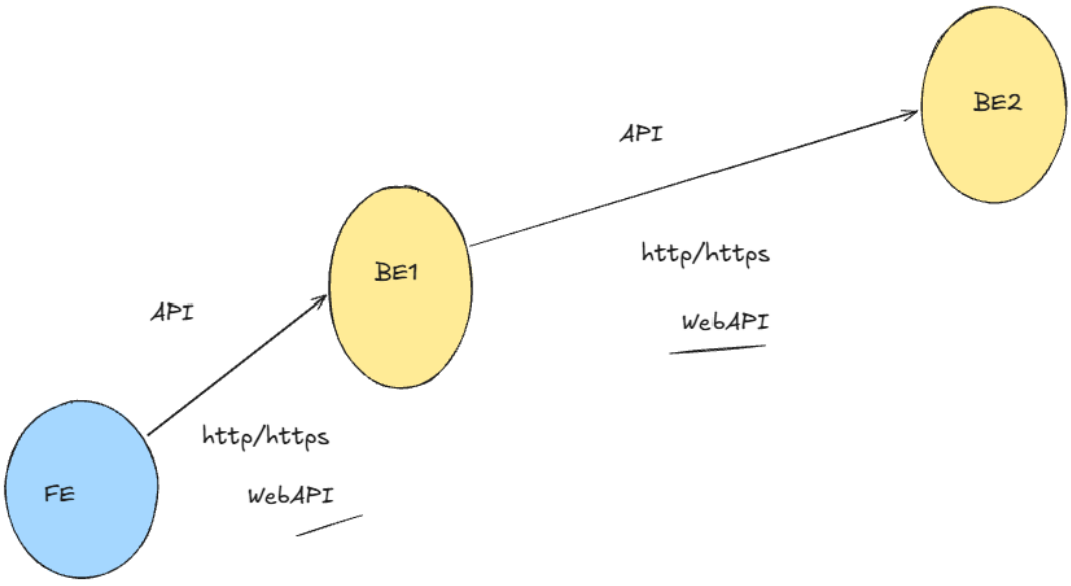
Further enhancement, froent-end separate project, backend separate project



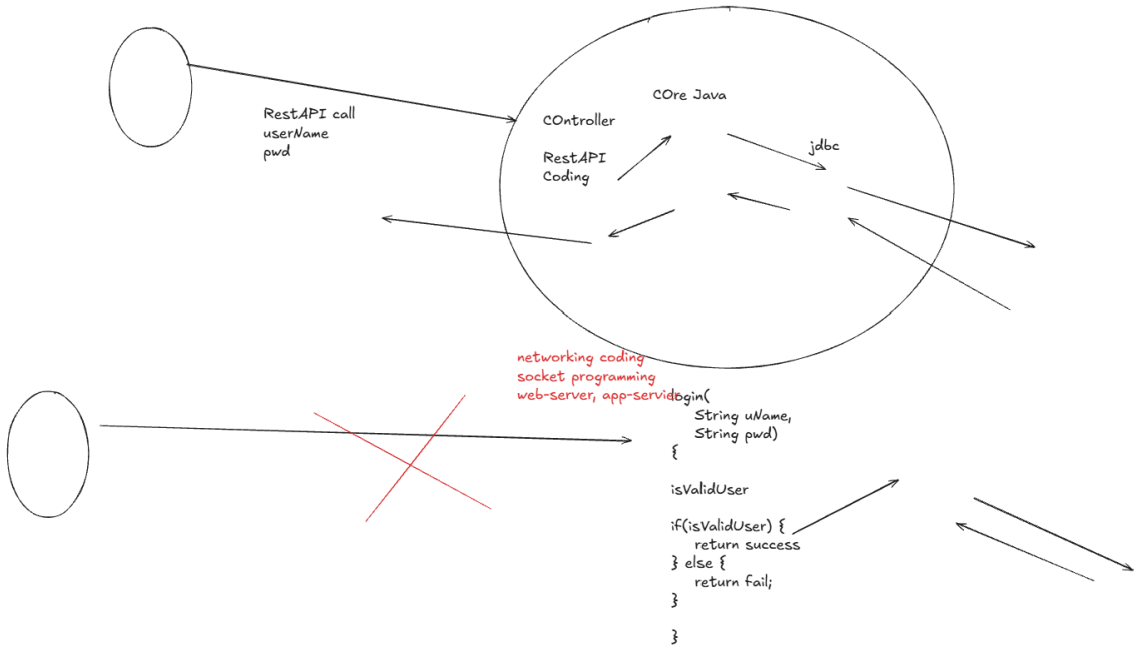
End-to-end flow - app deployed on AWS, and being used from client browser



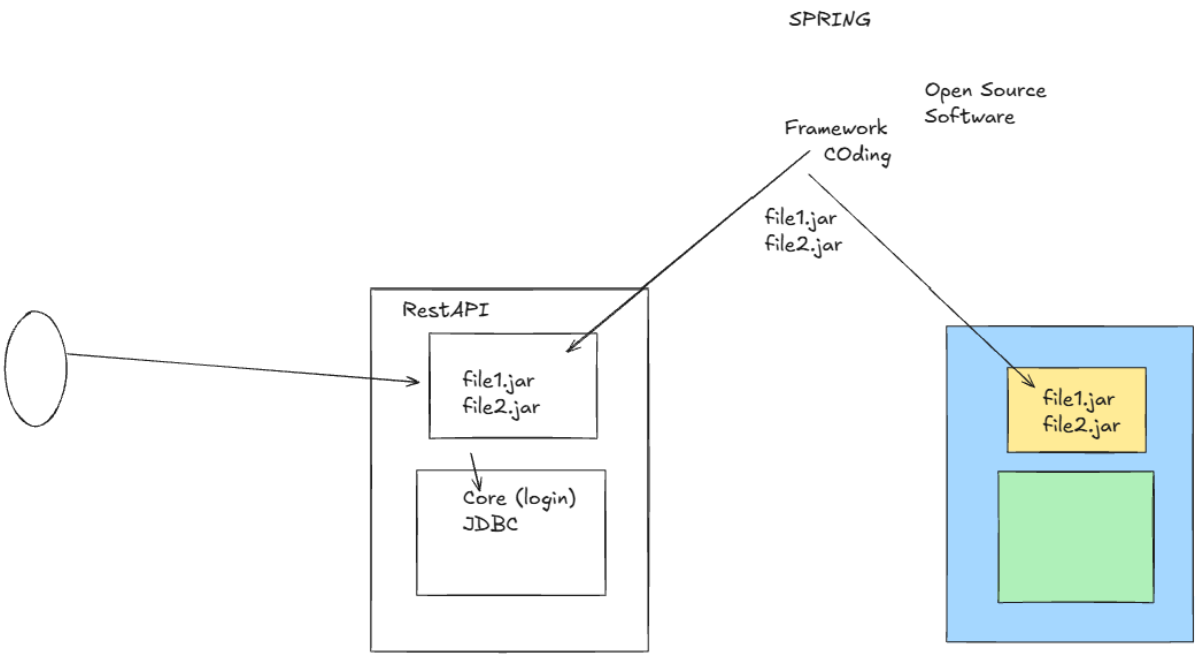
API - WebAPI



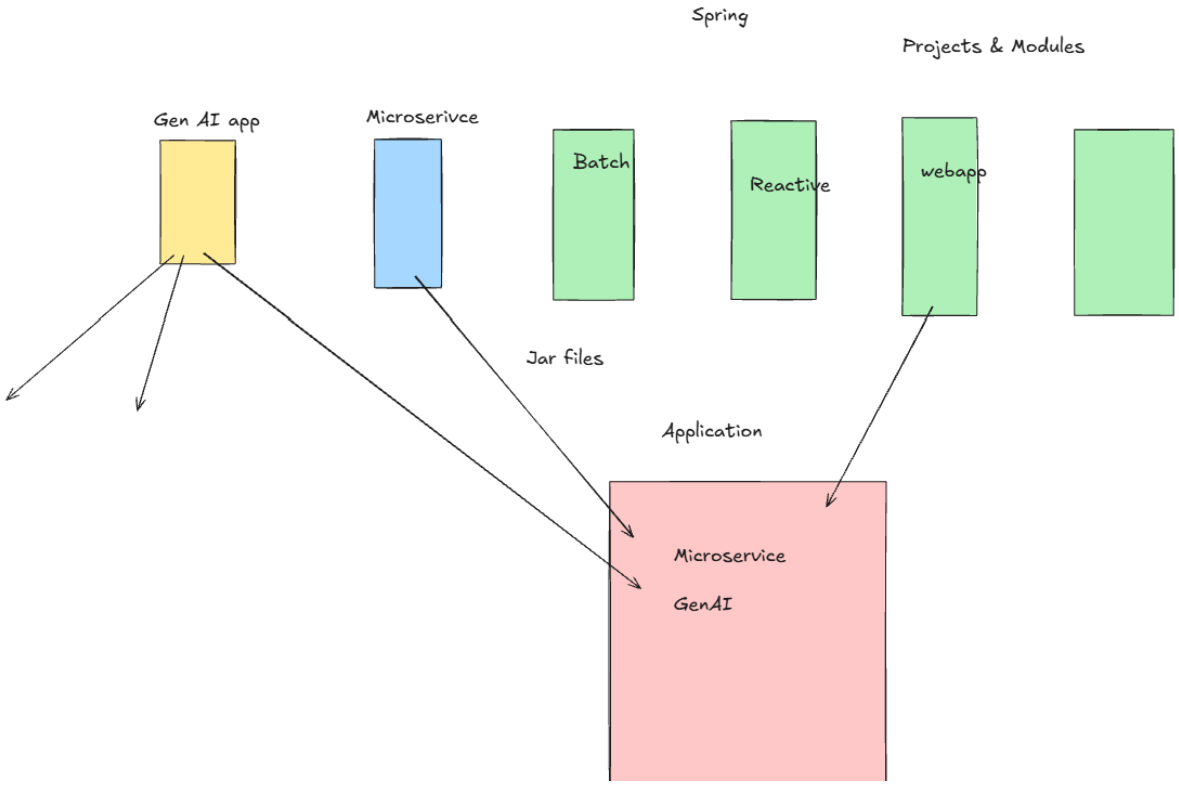
To expose your logic, so other services/fe can call your system you need to write much more logic - socket programming, networking.. So app logic + additional massive logic needed



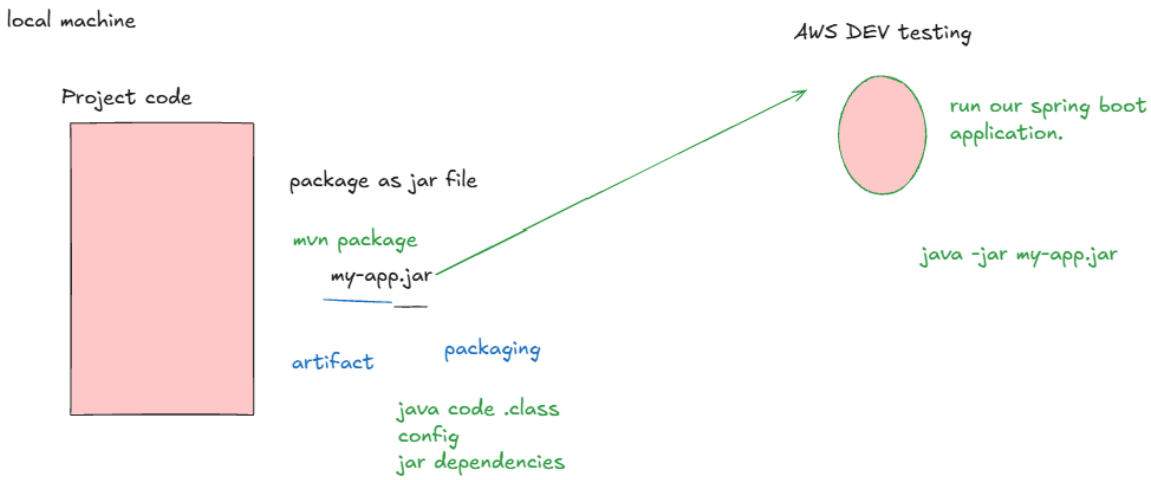
So we can use framework here: common logic written by framework, we just get it in local.



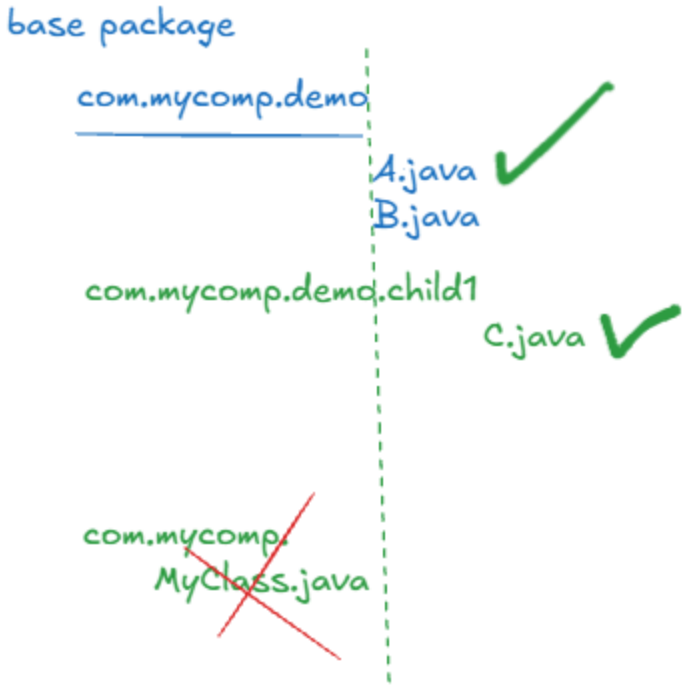
Using spring you can build any kind of application. The spring code is organized as project & modules. We need to choose right project & modules and add to our application to build that kind of application



Code in local, build jar, deploy & execute in AWS



All java classes under Base package or its sub packages



Properties & ymal setup

`application.properties`

properties

```
# Application
spring.application.name=payment-service
server.port=8080

# Database
spring.datasource.url=jdbc:mysql://localhost:3306/paymentdb
spring.datasource.username=root
spring.datasource.password=root123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
```

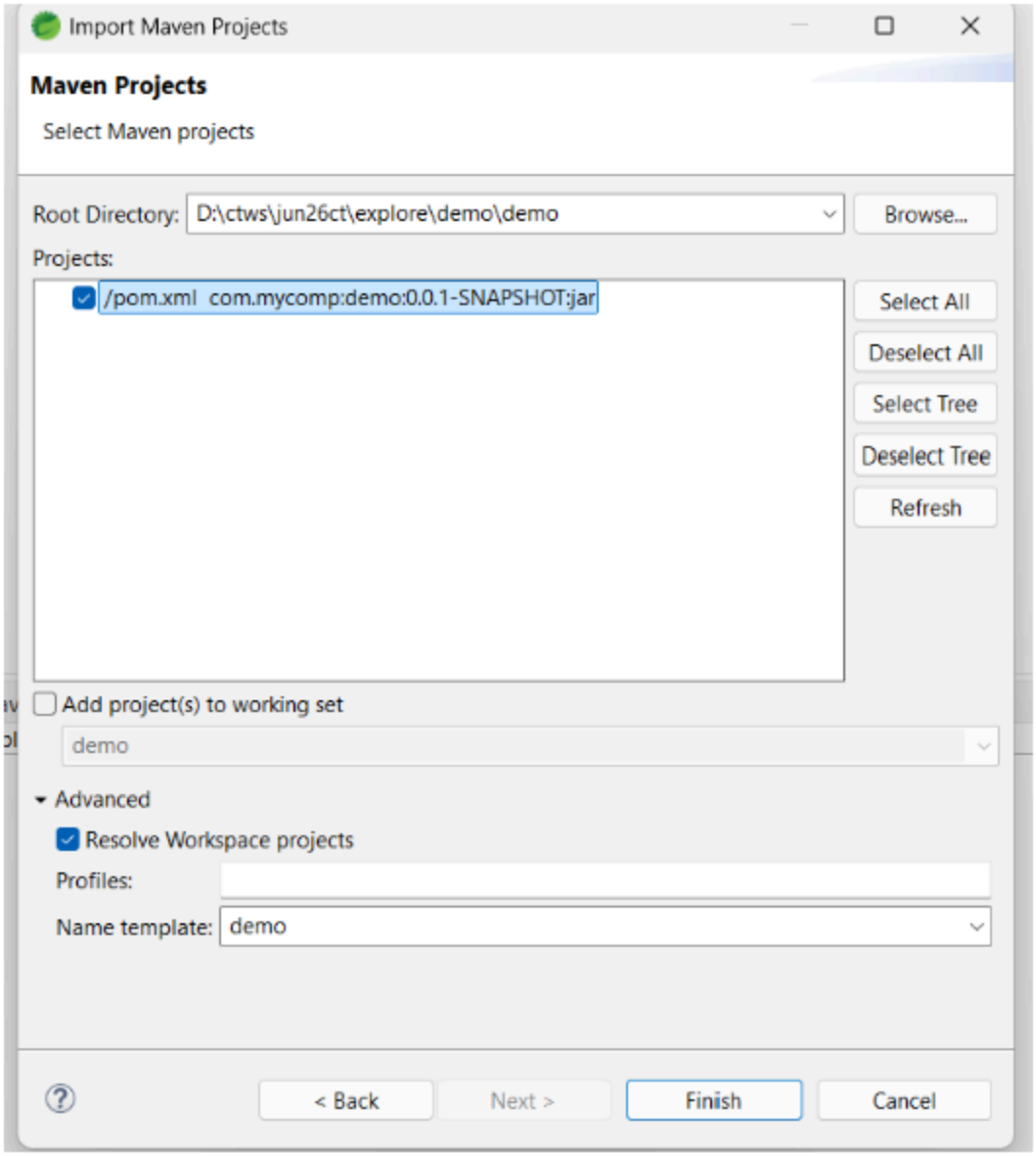
Equivalent `application.yml`

YAML

```
spring:
  application:
    name: payment-service

  datasource:
    url: jdbc:mysql://localhost:3306/paymentdb
    username: root
    password: root123
    driver-class-name: com.mysql.cj.jdbc.Driver
```

Importing demo app in IDE



Spring boot app structure that we got from start.spring.io

Building 1st spring boot application.
==========

Website - Webapplication.

Frontend - Backend.

=======

1. FE & BE application should be developed as separate projects
2. these separate projects will be deployed & executed independently in cloud (AWS)
3. Your application services will not be directly exposed to the outer world. it will be exposed via

some entry level software Loadbalancer, APIGateway, NginX,...
4. in DNS, we will have domain mapped
    ourdomain.com <IP of the entry level software>
5. When you run an application, it will expose/occupy some ports.
    your entry level software will expose 443 (https) & 80 (http) connection
6. in browser when you launch the domain, browser gets the mapped IP from DNS. and further connects to the entry level software
7. ROuting rules are configured at entry level, which will decide whether to give request to FE or backend. accordingly if its FE, request goes to FE
8. Accordingly req is processed and response is sent back.

==========

API - APplication Programming INterface
    2 parts of the systems can talk to each other using API

WebAPI - when 2 parts communicate over the web
    http/https protocol
    Webservice

how to develop webapis.
    there are guidelines provided.
    in the form of
    Rest
    SOAP

RestAPI
    rest principles, or rest guidelines are available
    when you develop an API using Rest guidenlines
    RestAPI
    Restful webservice
    Restful api

SOAP based API

======
API
Webapi, Webservice

RestAPI/Restful Webservice
SOAP API
=======

Build RestAPI
Frontend app => talk to backend application using rest API.

Framework - Common logic, which majority of app needs.
its already coded, and tested,
we can directly use this in our application
we don't have to develop it by ourselves

what is framework, why we need framework?

Spring is the powerful framework in Java communication.
Using Spring you can build any kind of application.

Spring has organized all of its common code in the form of projects & modules(some part of a project)

Depending about what application you are building, you need to add right project & module in your application.
====

About Spring Framework
=====

all the code which spring community has developed is available as Projects & Modules.

initiall
20+ years ago
https://spring.io/projects/spring-framework#overview

Rod Johnson

Code available in GitHub
https://github.com/spring-projects/spring-framework

https://github.com/spring-projects/spring-framework/blob/main/spring-web/src/main/java/org/springframework/http/ContentDisposition.java

jars - dependencies

Build Tool: Maven

Spring Boot - Rapid application development
    Using spring boot you can build any of the spring capable project in rapid manner.

Spring Boot Starters.

==========
- Spring keeps the code organized as "projects & modules" in GitHub. Open source.
- For our project we need to take this spring code (as jars)
=> You can do that using
    - Maven - build tool
    - Spring boot starters feature
==========

One microservice application, which will expose RestAPI to add 2 numbers.

Spring Boot

we can build our 1st spring boot application using spring initializr

https://start.spring.io/

any JVM based language can be used for building your spring boot app
    java - kotlin - groovy

build tools: maven(mostly adopted), gradle

spring boot latest version.

group: helps to identify which company is building the software
google.com

group: com.google

mycomp.com

group: com.mycomp

====

group

artifact: the final output file that will be generated by your project. (it will represent your project code)

base package: all the java code you do, should belong in the base package or subpackages under base package.

Config file can be either properties or Yamal file
---
in the project we might need to configure some key, value setup. So we can use either properties file or yaml file. both are structurally different, but doing the configuration itself.
Basicaly
KEY=VALUE

==========

1. Go to start.spring.io & create your 1st spring boot application. Spring Web dependency - RestAPI

2. Download the project.

3. Setup an IDE - Eclipse STS

java 17
from command prompt
java --version
install IDE

D:\ctws\jun26ct

D:\ctws\jun26ct\ide

4. the demo project downloaded from start.spring.io, we want to import into IDE.
unzip the demo under explore folder for your project base location.

- import into IDE

5. Explore the project created from star.spring.io & understand what is what.

6. few commonly used maven command.
Code a RestAPI to add 2 numbers.

Compilation---
mvn clean
mvn package

mvn clean package

Execution--
mvn spring-boot:run

if you have jar file
java -jar <filename>.jar

Core java application----------

code
javac - compilation
java - execution

-- TEAM we will resume shortly --
The LIVE will be ON. I'll sent a reminder in WhatsApp group, when we are starting with the topic.
may be 5-10mins more will be needed.

I would suggest you to also create your first spring boot app, import to IDE.

SINCE WE HAVE SOME TIME HERE

https://start.spring.io/
https://spring.io/tools#eclipse

Code

Compile
mvn clean
mvn package

IDE
clean
delete target folder
D:\ctws\jun26ct\explore\demo\demo\target\

package
- it compiled main java classes
- test classes
- config file was moved to target location

- ran testcases

- build file artifact jar

Executing

spring-boot:run

taskkill /PID 27164 /F

==========

Work with our 1st spring boot application

Code
start.spring.io

Compilation
use maven commands

mvn clean package

Execution
java -jar demo-0.0.1-SNAPSHOT.jar
mvn spring-boot:run
IDE GOAL
clean spring-boot:run

rt click => runas => spring boot application

on the main class, run as java application.

one service can occupy only one port.

default port is 8080
2 times when you try to run the application on same port, then it will fail.
you need to find the existing execution & stop it
Default port is 8080
when you run the service 1 time, it will occupy 8080

if you try to run the same service again, it will try to occupy 8080 again. 2nd time it will fail.

if you have engaged with target folder, and doing clean, that might fail.

GitHub copilot code rest API
Lombok
devtools
Slf4j with logback

work with AWS deploying our application in AWS.

working with profile - Spring profilt.

==========

Framework
Power of Spring framework
Build your 1st spring boot application
build & execute it in local IDE.

==========